

save the changes. You should see your picture in the top left corner of the screen.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the GitLab System Admin Hands-on Guide, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/sysadminhandsonlab3.md

> Estimated time to complete: 40 minutes

Objectives

The objective of this lab is to demonstrate how to create and register a GitLab Runner on a server. For more information, see the [GitLab Runner documentation](#).

Task A. Install the gitlab-runner package

1. Use your assigned IP address and SSH key file to log into your GitLab Runner server (not your Omnibus server).

```
bash
ssh -i <YOURASSIGNEDSSHKEYFILE> root@<YOURRUNNERSERVERPUBLICIP>
```

1. If your system displays an authentication warning, type yes and press <kbd>Enter</kbd>

1. Add the GitLab Runner installation repository.

```
bash
curl -L "https://packages.gitlab.com/install/repositories/runner/gitlab-runner/script.deb.sh" | sudo bash
```

1. Install the GitLab Runner agent.

```
bash
sudo apt-get install -y gitlab-runner
```

1. Once the install completes, check that the service is running.

```
bash
sudo gitlab-runner status
```

Task B. Register a runner with GitLab

1. Sign into your GitLab instance with a web browser and select Admin Area from the left sidebar.

1. In the left sidebar, under CI/CD, select Runners.

1. Select New instance runner.

1. In the Tags section, select Run untagged jobs.

> This will allow the runner to pick up any jobs rather than just jobs with specific tags.

1. Leave all other options as default and select Create runner.

1. Ensure that Linux is selected as the Operating system for the runner.

1. Copy the command in Step 1, and add sudo in front of the command. Run it in your command prompt.

1. The command will first prompt you for your GitLab instance URL. Verify that this URL matches your GitLab instance, then press `<kbd>Enter</kbd>`

1. Enter any appropriate name for your runner.

1. When you are prompted for an executor, type shell.

> A shell executor will run jobs using your instance's shell. For more information about executors, check the documentation.

1. Run `sudo gitlab-runner list` to verify the runner after registration.

1. Return to your web browser and select View runners. Verify the runner you registered appears in the list and shows as online.

Task C. Test the runner with a CI/CD pipeline

1. Navigate back to the homepage of your GitLab instance by selecting the GitLab icon at the top of the left sidebar.

1. Select Create a project.

1. Select Create blank project.

1. Enter CICD Test as the project name.

1. In the Project URL, select root from the namespace dropdown.
1. Leave all other settings as they are and click Create project.
1. In the middle of the project landing page, under the project title, select the + dropdown. Select New file.
1. On the new file page, enter .gitlab-ci.yml as the file name.
1. Paste the following code into the body of the file.

```
yml
stages:
  - build
  - test

buildapp:
  stage: build
  script:
    - echo "The build stage requires at least one job"

testapp:
  stage: test
  script:
    - echo "The test stage requires at least one job"
```

1. Select Commit changes.
1. In the left sidebar, select Build > Pipelines.
1. Select the pipeline status (it should say passed).
1. Click into each of the buildapp and testapp jobs to see the job logs and commands that were executed on the runner.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the GitLab System Admin Hands-on Guide, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/sysadminhandsonlab4.md

> Estimated time to complete: 30 minutes

Objectives

The objective of this lab is to demonstrate how to back up a GitLab instance on a virtual machine, and restore the instance to a previous state.

For more information, see documentation about backing up and restoring a GitLab instance.

Task A. Configure backup settings

1. Open an SSH session on your GitLab instance server.

1. Search for the location of backup settings in gitlab.rb.

```
bash
sudo grep -n backuppath /etc/gitlab/gitlab.rb
```

1. Note the line number for the setting gitlabrails['backuppath'].

1. Create a new directory to hold GitLab backups.

```
bash
sudo mkdir /tmp/backups
```

1. Edit gitlab.rb to change the backup path. Replace "606" with the line number noted in step 3.

```
bash
sudo sed -i '606s@/var/opt/gitlab/backups@/tmp/backups@' /etc/gitlab/gitlab.rb
sudo sed -i '606s///' /etc/gitlab/gitlab.rb
```

> Here, we are using the sed command to do text replacements inside the gitlab.rb file without having to use a text editor like vim.

1. Reconfigure to apply the changes.

```
bash
sudo gitlab-ctl reconfigure
```

Task B. Backup the GitLab instance

1. Take a full backup of the GitLab instance.

```
bash
sudo gitlab-backup create
```

1. After the backup completes, go to the backup location and inspect the backup file.

```
bash
sudo ls /tmp/backups
sudo tar -tvf /tmp/backups/<backupfilename>
```

Task C. Make some changes to GitLab settings

1. Sign into your GitLab instance with a web browser and open your sidebar. In the bottom left corner, click Admin area.
2. In the left sidebar, select Settings > General.
3. Expand Account and limit and change the maximum attachment size to 500 MiB, and the default project limits to 10000.
4. Click Save changes to save the changes.
5. Refresh the page and verify your changes were applied.

Task D. Restore from backup

1. Return to the SSH session on your GitLab instance server.
1. Move your backup file to the location GitLab requires for performing the restore.

```
bash
sudo cp /tmp/backups/<backupfilename> /var/opt/gitlab/backups/
```

1. Ensure the backup file has correct permissions for performing the restore.

```
bash
sudo chown git:git /var/opt/gitlab/backups/<backupfilename>
```

1. Stop the puma and sidekiq services before restoring.

```
bash
sudo gitlab-ctl stop puma
sudo gitlab-ctl stop sidekiq
sudo gitlab-ctl status
```

1. Restore from backup. Replace <backuptimestamp> with the portion of the backup filename up to and including -ee. For example, if the backup file name starts with 16632077322022091515.3.3-ee, the command will be `sudo gitlab-backup restore BACKUP=16632077322022091515.3.3-ee`.

```
bash
sudo gitlab-backup restore BACKUP=<backuptimestamp>
```

1. Type yes when prompted during the restore operation. You may see what looks like error messages. That is normal.

1. When prompted to rebuild the authorizedkeys file, type yes.

1. Restart sidekiq and puma services.

```
bash
sudo gitlab-ctl start sidekiq
sudo gitlab-ctl start puma
sudo gitlab-ctl status
```

1. Wait up to 5 minutes before refreshing GitLab in your web browser. Verify that the maximum attachment size and the default project limits you changed revert back to the defaults (i.e. when the backup was taken).

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the GitLab System Admin Basics Hands-on Guide, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/sysadminhandsonlab5.md

> Estimated time to complete: 30 minutes

Objectives

This lab will help you improve your instance's security by enabling the option to send a confirmation email on signup, and ensure that signups are only allowed from your company's domain. For more information, see [sign-up restrictions documentation](#).

Task A. Add Sign-up Restrictions

1. Log into your GitLab web instance with your root user and password from Lab 1.

1. In the bottom left corner of the main screen in the sidebar, click Admin Area.

1. In the bottom of the left hand side navigation pane and click Settings > General.
1. Under Sign-up restrictions, click Expand.
1. Under Email confirmation settings, click the radio button next to Hard.
1. Next, ensure sign ups are only allowed for your company's domain. In the Allowed domains for sign-ups, type your company's domain name and press `<kbd>Enter</kbd>`.
1. Scroll down to the end of the section and click Save Changes.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the GitLab System Admin Basics Hands-on Guide, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/sysadminhandsonlab6.md

> Estimated time to complete: 30 minutes

Objectives

The objective of this lab is to guide you on managing your GitLab logs by using the `gitlab-ctl` and `sed` commands. For more information, see the GitLab logging documentation.

Task A. View active logs

The `gitlab-ctl` command allows you to tail all GitLab log files as well as filter by GitLab service.

1. From a shell session on your GitLab instance, run the following command to view all active GitLab logs.

```
bash
sudo gitlab-ctl tail
```

Amidst all the output, you should notice the command shows the full file path to each log. Most GitLab logs live in `/var/log/gitlab`. (Note: You can type CTRL-C to exit the tail command.)

1. You can also view GitLab logs by service. Run the following command to view only NGINX logs (i.e. log files in `/var/log/gitlab/nginx`).

```
bash
sudo gitlab-ctl tail nginx
```

You should now see the most recent entries of log files specific to the NGINX web server.

1. Finally, you can drill down to an individual log file.

```
bash
sudo gitlab-ctl tail nginx/gitlabaccess.log
```

Task B. Set minimum log levels

Admins are able to set minimum log levels for some GitLab services. Note that only some services such as NGINX and Gitaly let admins change the minimum logging level, and even then only for some log files. The loglevel for other services, such as Sidekiq and Redis, cannot be changed.

1. Check the current minimum log levels for GitLab services.

```
bash
sudo grep -n -E 'loglevel|logginglevel' /etc/gitlab/gitlab.rb
```

1. Note the line number for nginx['errorloglevel'].

1. Change the minimum log level for nginx. Replace "1731" with the appropriate line number from the grep output in the previous step.

```
bash
sudo sed -i '1731s/\["error"\]/["warn"]/' /etc/gitlab/gitlab.rb
sudo sed -i '1731s/ //' /etc/gitlab/gitlab.rb
```

1. Re-run the grep command from Step 1 to verify the line was modified as intended.

1. Reconfigure to apply the changes.

```
bash
sudo gitlab-ctl reconfigure
```

Task C. Manage log retention

GitLab uses logrotate to manage retention of all logs except those managed by the runit service manager (runit uses a separate service logging daemon called svlogd). Log retention can be configured in /etc/gitlab/gitlab.rb.

1. Examine default logrotate retention settings.


```
bash
sudo grep -n 'logrotate' /etc/gitlab/gitlab.rb
```

1. Optional: View the default retention settings for the runit-managed logs.

```
bash
sudo grep -n 'svlogd' /etc/gitlab/gitlab.rb
```

1. It appears logrotate (and svlogd) rotate log files every day, and retain 30 days worth of logs. We can verify this by looking inside the service log directories.

```
bash
sudo ls /var/log/gitlab/puma
```

Note the gzipped archive files for Puma's stdout and stderr logs from previous days.

1. Change logrotate's behavior to rotate log files weekly. As before, modify the line sed edits accordingly using the line number from the grep output.

```
bash
sudo sed -i '1234s/daily/weekly/g' /etc/gitlab/gitlab.rb
sudo sed -i '1234s/ //' /etc/gitlab/gitlab.rb
```

1. Change logrotate's retention period to 1 year of retained log files. As before, modify the line sed edits accordingly using the line number from the grep output.

```
bash
sudo sed -i '1234s/30/52/g' /etc/gitlab/gitlab.rb
sudo sed -i '1234s/ //' /etc/gitlab/gitlab.rb
```

1. Run the following again to ensure your changes are properly written to gitlab.rb.

```
bash
sudo grep -n 'logrotate' /etc/gitlab/gitlab.rb
```

1. Reconfigure to apply the changes.

```
bash
sudo gitlab-ctl reconfigure
```

Task D. Change log formatting

Many logs are JSON formatted by default. Admins may wish to configure text formatting depending

on the log ingestion system used, or for readability.

1. Check the current log formats for Gitaly.

```
bash
sudo grep -n -F "gitaly['configuration']" -A20 /etc/gitlab/gitlab.rb
```

> This command will find the start of the gitaly configuration with grep, then display the 20 lines that follow using the -A20 flag. This will show the full configuration of the Gitaly configuration file

1. Within the output, locate the following lines and note their line numbers

```
bash
gitaly['configuration'] = {
...
  logging: {
    dir: "/var/log/gitlab/gitaly",
    level: 'warn',
    format: 'json',
    sentrydsn: 'https://<key>:<secret>@sentry.io/<project>',
    sentryenvironment: 'production',
  },
```

1. Run `sudo gitlab-ctl tail gitaly/current` to see the current JSON output for Gitaly logging.

1. Change Gitaly's log format from JSON to text formatting. Ensure you align the line numbers to the correct lines from above.

```
bash
sudo sed -i '2588s/json/text/' /etc/gitlab/gitlab.rb
sudo sed -i '2574s/ //' /etc/gitlab/gitlab.rb
sudo sed -i '2588s/ //' /etc/gitlab/gitlab.rb
sudo sed -i '2585s/ //' /etc/gitlab/gitlab.rb
sudo sed -i '2591s/ //' /etc/gitlab/gitlab.rb
sudo sed -i '2591s/,/ }/' /etc/gitlab/gitlab.rb
```

> With these sed commands, you are first replacing the JSON format with text. Next, you are removing the comments in front of the format and Gitaly configuration blocks to enable them.

1. Rerun your grep command to view your configuration: `sudo grep -n -F "gitaly['configuration']" -A20 /etc/gitlab/gitlab.rb`. The end result will look similar to below:

```
bash
gitaly['configuration'] = {
...

```

```
logging: {  
  dir: "/var/log/gitlab/gitaly",  
  level: 'warn',  
  format: 'text'  
  sentrydsn: 'https://<key>:<secret>@sentry.io/<project>',  
  sentryenvironment: 'production',  
}
```

1. Reconfigure to apply the change.

```
bash  
sudo gitlab-ctl reconfigure
```

1. Verify the updated formatting.

```
bash  
sudo gitlab-ctl tail gitaly/current
```

You should see the log output is now text formatted instead of JSON formatted.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the GitLab System Admin Basics Hands-on Guide, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/sysadminhandsonlab7.md

> Estimated time to complete: 30 minutes

Objectives

The purpose of this lab is to walk you through using Prometheus Service, and how to view Prometheus metrics. For more information, see documentation about monitoring GitLab with Prometheus.

Task A. Access the Prometheus Service

1. GitLab's Prometheus server can be reached via TCP port 9090. Unfortunately, the training environment currently blocks inbound traffic to that port. As a workaround, you can open an SSH tunnel as follows.

```
bash
```

```
ssh -L 9090:localhost:9090 -i <SSHKEYNAME> root@<GITLABINSTANCEHOSTNAME>
```

1. Navigate to <http://localhost:9090> in a web browser to view the built-in Prometheus server.

1. To view the available metrics in Prometheus, navigate to <http://localhost:9090/metrics>.

> To see a more detailed list of each metric, check out the documentation.

Task B. Viewing Prometheus Metrics

> For this example, suppose you wanted to monitor HTTP requests sent into your GitLab instance. To achieve this, we can use some built in GitLab metrics.

1. Navigate to <http://localhost:9090>.

1. In the input beside the magnifying glass button, type `httprequeststotal`.

1. Click the Execute button.

1. Click Table to view a table of all the results generated by the query.

> You will likely see a variety of different HTTP requests in the results. Note that you can see the job, instance, HTTP method and status of each request.

1. Click Graph to view a graph of your data.

> On the left hand side of the graph, you will see a - and + button. You can click these buttons to change the scale of the graph. Try adjusting the scale to see how it impacts your data.

Task C. Alert Configurations

> Prometheus can be used to alert administrators based on specific metrics. GitLab comes with a set of predefined alerts for Prometheus.

1. Navigate to <http://localhost:9090>.

1. Click Alerts in the top menu.

> Note that there are two sets of alert rules, one named GitLab and one named Node. The GitLab alerts correspond to platform issues such as a service being down, Postgres being down, or high queueing within services. A Node alert corresponds to the server GitLab is deployed on.

1. Note the file that contains the alerting rules. By default, the file is `/var/opt/gitlab/prometheus/rules/gitlab.rules`.

> As an administrator, you can define custom rules inside this file to use for alerting. This will allow you to monitor your system for any potential issues.

Task D. Common Performance Metrics

There are a variety of different metrics administrators can use to monitor the performance of their GitLab instances. Let's examine a few common queries you can use to monitor an instance.

1. Navigate to <http://localhost:9090>.

1. In the input beside the magnifying glass button, type the following query:

```
text
((nodememoryMemAvailablebytes / nodememoryMemTotalbytes) or ((nodememoryMemFreebytes +
nodememoryBuffersbytes + nodememoryCachedbytes) / nodememoryMemTotalbytes)) 100
```

> This query will show you the percentage of memory available on your instance.

1. Click Graph to view the percentage of memory over time.

1. In the query input, delete your previous query and replace it with the following query:

```
text
1 - avg without (mode,cpu) (rate(nodecpusecondstotal{mode="idle"}[5m]))
```

> This query will show you the percentage of CPU used on your instance.

1. Click Graph to view the percent of CPU used over time.

> For these metrics and many others, you may want to consider alerting for specific conditions. For example, you could alert for cases where the CPU usage exceeds 50% consistently, as it might indicate an issue.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the GitLab System Admin Hands-on Guide, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/sysadminhandsonlab8.md

> Estimated time to complete: 30 minutes

Objectives

The purpose of this lab is to show how to troubleshoot the GitLab server by using the `gitlab-ctl` command. For this lab exercise, refer to GitLab's application architecture to review GitLab's major services and interactions.

Task A. Troubleshoot NGINX

1. From a shell session on your GitLab instance, view one of the NGINX active logs.

```
bash
sudo gitlab-ctl tail nginx/gitlabaccess.log
```

Note the log adds new entries every few seconds. Most of these entries are `gitlab-runner` checking in with the GitLab instance via HTTP.

1. Stop the NGINX services.

```
bash
sudo gitlab-ctl stop nginx
```

1. Attempt to navigate to `http://<yourgitlabinstance>` using a web browser. Your web browser should display "This site can't be reached" or a similar message.

1. Check `nginx/accesslog` again.

```
bash
sudo gitlab-ctl tail nginx/gitlabaccess.log
```

The log should no longer be updating since no clients can make HTTP/HTTPS requests to GitLab after stopping NGINX.

1. Verify web services aren't running or listening anywhere.

```
bash
curl -i http://localhost/nginxstatus
curl -i http://localhost:80
```

1. Restart NGINX services.

```
bash
sudo gitlab-ctl restart nginx
```

1. Verify the clients (e.g. the GitLab Runner) can communicate with GitLab again.

```
bash
sudo gitlab-ctl tail nginx/gitlabaccess.log
```

1. Verify the webserver is running and listening on port 80.

```
bash
curl -i http://localhost/nginxstatus
```

Task B. Troubleshoot Puma

1. Connect to your GitLab instance with a web browser. Verify you can click around to projects, the Admin Area, etc.

1. From a shell session on the GitLab instance, stop Puma.

```
bash
sudo gitlab-ctl stop puma
```

1. Refresh GitLab in your web browser. You should immediately see an error that reads "502: GitLab is taking too much time to respond". NGINX is running, so it can accept HTTP requests. However, when workhorse tries to pass an HTTP request to the Rails application, there is no running service to accept it.

1. View the GitLab Workhorse logs.

```
bash
sudo gitlab-ctl tail gitlab-workhorse/current
```

You will see a variety of 502 and badgateway errors in the output.

1. View Puma logs.

```
bash
sudo gitlab-ctl tail puma
```

You should see a message in puma/pumastdout.log about the Puma service shutting down. You may also see errors in puma/pumastderr.log.

1. Restart Puma.

```
bash
sudo gitlab-ctl restart puma
```

1. View Puma's runit log.

```
bash
sudo gitlab-ctl tail puma/current
```

You may see output indicating Puma has restarted.

1. View puma/pumastdout.log.

```
bash
sudo gitlab-ctl tail puma/pumastdout.log
```

You should see that Puma is running and consuming resources again.

1. Wait about 2 minutes, then refresh GitLab in your web browser. The application should now be reachable.

1. View the GitLab Workhorse log.

```
bash
sudo gitlab-ctl tail gitlab-workhorse/current
```

Recent entries should indicate successful requests to Puma (i.e. when you reloaded GitLab in your web browser).

Task C. Troubleshoot Gitaly

1. Connect to your GitLab instance with a web browser.

1. Navigate to Menu > Projects > Your Projects.

1. Select New Project.

1. Select Create blank project.

1. Name the project Test project. Set project visibility to Public, and ensure Initialize repository with a README is selected. Leave all other settings as they are.

1. Select Create project. You will be redirected to the project's landing page.

1. SSH into your GitLab Runner server.

```
bash
ssh -i <SSHHOSTKEY> root@<GITLABrunnerhost>
```

1. Download Git if it is not already installed.

```
bash
sudo apt-get install -y git
```


1. Back on GitLab's Test project, select Clone on the right side of the page.
1. Next to Clone with HTTP, select Copy URL.
1. From your GitLab Runner server, clone the repository.

```
bash
git clone <URLcopiedfrompreviousstep>
```

1. Verify the project is correctly cloned.

```
bash
cd ~/test-project
ls -a
git status
```

1. Enter exit to exit the SSH session on your GitLab Runner server.
1. Open an SSH session on your GitLab Omnibus instance.

```
bash
ssh -i <SSHHOSTKEY> root@<GITLABOMNIBUSHOST>
```

1. Verify Gitaly is running.

```
bash
sudo gitlab-ctl status gitaly
```

1. View Gitaly logs.

```
bash
sudo gitlab-ctl tail gitaly
```

> You should see many recent gRPC requests relating to Test Project (you can see the references more clearly if you grep the output, e.g. `sudo gitlab-ctl tail gitaly | grep test-project`).

1. Stop Gitaly services.

```
bash
sudo gitlab-ctl stop gitaly
```

1. Verify Gitaly (and only Gitaly) is stopped.

```
bash
sudo gitlab-ctl status
```

1. Navigate back to Test Project in your web browser. On the project page select the dropdown that says main under the project title. Ordinarily you would be able to select a Git branch to switch to. Now you see an error and the branch list will not load.

1. In the left sidebar, select Repository > Files. Note the 404 error as GitLab is unable to fetch any repository files.

1. Select the Back button to go back to the project landing page. Then refresh the page.

1. Note the additional errors. One error may read "An error occurred while fetching folder content". GitLab cannot checkout the HEAD of the default branch because Gitaly is not running to handle the request.

1. Return to your GitLab instance SSH session. Check Gitaly's recent log entries.

```
bash
sudo gitlab-ctl tail gitaly/current
```

> Note the many errors in the log output.

1. Enter exit to exit the SSH session on your GitLab Instance server.

1. SSH back into your GitLab Runner server.

```
bash
ssh -i <SSHHOSTKEY> root@<GITLABRUNNERHOST>
```

1. Navigate into your cloned Test Project.

```
bash
cd ~/test-project
```

1. Try to fetch any new changes from the remote repo on the GitLab instance.

```
bash
git fetch
```

> You may see error 503 in the output, indicating Gitaly is not reachable and can not handle the request.

1. Enter exit to exit the SSH session on your GitLab Runner server.

1. Re-initiate an SSH session on your GitLab Omnibus instance.

```
bash
ssh -i <SSHHOSTKEY> root@<GITLABOMNIBUSHOST>
```

1. Restart Gitaly services.

```
bash
sudo gitlab-ctl start gitaly
```

1. Check Gitaly logs.

```
bash
sudo gitlab-ctl tail gitaly/current
```

> The output should now show successful gRPC requests.

1. Return to Test Project in your web browser. Refresh the page. You should now be able to navigate around the repository, view files, and check out branches.

1. SSH back into your runner server and test git fetch. The command should now run without errors (there probably will not be any output since files have not changed in GitLab).

Task D. Run the gitlabsos utility

1. Navigate to the gitlabsos project page. Read through the project summary and README to learn the utility's purpose and usage.

2. Connect to your GitLab Omnibus instance via SSH.

3. Clone the gitlabsos utility.

```
bash
/opt/gitlab/embedded/bin/git clone --recursive https://gitlab.com/gitlab-
com/support/toolbox/gitlabsos.git
```

4. Run gitlabsos.

```
bash
cd gitlabsos
sudo ./gitlabsos.rb
```

> The script may take a few minutes to run.

5. Once the script is finished, examine the resulting report file and its contents.

```
bash
ls
tar -tvf gitlabsos.<GITLABFQDN>.<timestamp>.tar.gz
```

GitLab Support may ask for this report to assist with troubleshooting.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the GitLab System Admin Hands-on Guide, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/_index.md

Current Offerings

The GitLab Professional Education Services team currently provides the following offerings:

- Live Instructor-led training (ILT)
- GitLab Technical Certifications as Professional Service offerings.
- Train-the-Trainer

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/advgitlababcdhandson.md

GitLab Advanced CI/CD Lab Guides

Lab Name	Lab Link
GitLab Runners Deep Dive	Lab Link
Optimizing Build Pipelines	Lab Link
Optimizing Pipeline Testing	Lab Link
Reviewing Best Practices	Lab Link
Configurations for Complex Processes	Lab Link
Review Apps	Lab Link
Deployment Strategies	Lab Link

Quick links

Here are some quick links that may be useful when reviewing this Hands-On Guide.

GitLab CI/CD Course Description

GitLab CI/CD Specialist Certification Details

Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab Advanced CI/CD, please submit your changes via Merge Request!

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/advgitlabciacdhandsonlab1.md

Runner scalers rely on a consistent runner image to be able to spin up runner machines on demand. The first step to creating this image is understanding how the underlying runner functions. In this lab, you will learn how to create a Docker based runner. This runner will be able to serve as a basis for a Docker autoscaler.

> Estimated time to complete: 15 minutes

Task A. Create a new project

To start, let's create a new project in the lab environment:

1. Select Create a project.
1. Select Create blank project.
1. For the project name, type CICD Runner.
1. Leave all other options as default and select Create project.

Task B. Adding the runner to the project

1. Ensure that you are now looking at your newly created project's page.
1. In the left sidebar, select Settings > CI/CD.
1. Click the arrow next to Runners to expand the Runners section.
1. Select New project runner.
1. Select Run untagged jobs, leave all other settings as default, and select Create runner.
1. Ensure that Linux is selected as your operating system.
1. In the section titled Step 1, review the command, which will look something like this:

```
shell
gitlab-runner register
```

```
--url https://ilt.gitlabtraining.cloud
--token glrt-bzoxCnA6aDlvCnQ6Mwp1OmFtdCQKGI9glOywWMYcfTG74GwQ.1c1rc1xe9
```

1. Take note of the value following --token. You will need this token later for the registration of your runner.

1. Finally, select View Runners. You may see a warning message asking if you are sure you want to leave the page without saving. This is fine to do as long as you have made a note of your token.

Task C. Deploying a runner

We will manage the association of the runner and deployment of runner configuration through GitLab. This strategy allows you to have source control on your runner configuration, which is ideal for tracking changes.

Let's take a look at how this is structured:

1. First, let's add our token securely to our project as a project-level variable. Navigate to Settings > CI/CD and expand the Variables section. Select Add variable. In the pane on the right, type GITLABRUNNERTOKEN into the Key field, and paste your runner token into the Value field.

1. Navigate to your project repository.

1. Select + > New file.

1. In the filename, type .gitlab-ci.yml.

1. Add the deploy stage to the .gitlab-ci.yml file:

```
yml
stages:
  - deploy
```

1. Our first task is to set up our job to install the required dependencies for an SSH connection. Copy and paste the code below.

```
yml
deploy config:
  stage: deploy
  image: ubuntu:latest
  beforescript:
    - 'which ssh-agent || ( apt-get update -y && apt-get install openssh-client git -y )'
    - eval $(ssh-agent -s)
    - chmod 400 "$SSHPRIVATEKEY"
    - ssh-add "$SSHPRIVATEKEY"
    - mkdir -p ~/.ssh
```

- `chmod 700 ~/.ssh`
- `ssh-keyscan -t rsa,ed25519 $ip >> ~/.ssh/knownhosts`

> This job starts by installing and starting an ssh agent on the runner. When you redeemed your invitation code, an instance was created for you to deploy to and the SSH private key is stored in a variable named `SSHPRIVATEKEY`. This key is added to the SSH agent to use for connections.

1. As a script for the job, we are going to SSH into our server and register the gitlab runner on it. Notice that we are using the `GITLABRUNNERTOKEN` variable we created earlier.

```

yml
  script:
    - ssh root@$ip 'gitlab-runner unregister --all-runners'
    - ssh root@$ip 'gitlab-runner register --non-interactive --url
https://lt.gitlabtraining.cloud --executor "docker" --docker-image alpine:latest --token
"$GITLABRUNNERTOKEN"'

```

> The first command we run will unregister any current runners on your remote server. This prevents duplicate registrations of runners.

>

> The `--non-interactive` flag prevents the runner from prompting us for inputs during the installation process. Rather than entering a prompt, we provide the arguments for URL, executor, docker image, and token through command line arguments.

>

> In this configuration, the executor is set to docker. For the docker-image, you are setting the default Docker image to use for your pipelines. You can use any Docker image you like, for this example, we will use `alpine:latest` as the default image.

1. Select Commit changes, add a commit message (e.g. "Added runner creation job") and select Commit changes.

1. Select Build > Pipelines.

1. Select your most recent pipeline.

1. Verify that the pipeline completes successfully.

To verify that the runner is registered:

1. In the left sidebar, select Settings > CI/CD.

1. Click the arrow next to Runners to expand the Runners section. You should see a green circle next to your runner.

Task D. View your runner configuration

When this runner is created, it will have a `config.toml` file that defines the configuration of

the runner. Let's start by viewing our current runner configuration.

1. At the end of your deploy config job script, add the following command:

```
yaml
- ssh root@$ip 'cat /etc/gitlab-runner/config.toml'
```

1. Commit this change and navigate to the pipeline created from the change.

1. View the deploy config job output.

1. You will see an output from the cat command similar to below:

```
toml
concurrent = 1
checkinterval = 0
shutdowntimeout = 0

[sessionserver]
  sessiontimeout = 1800

[[runners]]
  name = "runner-test"
  url = "https://ilt.gitlabtraining.cloud"
  id = 1852
  token = "your-token-here"
  tokenobtainedat = 2025-05-08T12:59:30Z
  tokenexpiresat = 0001-01-01T00:00:00Z
  executor = "docker"
[runners.custombuildidir]
[runners.cache]
  MaxUploadedArchiveSize = 0
[runners.cache.s3]
[runners.cache.gcs]
[runners.cache.azure]
[runners.docker]
  tlsverify = false
  image = "alpine:latest"
  privileged = false
  disableentrypointoverwrite = false
  oomkilldisable = false
  disablecache = false
  volumes = ["/cache"]
  shmsize = 0
  networkmtu = 0
```

Task E. Editing your runner configuration

For our configuration, we want to be able to run our job with Docker in Docker. To do this, we need to change two default configurations:

```
toml
privileged = true
volumes = ["/certs/client", "/cache"]
```

To make these changes, we will push a config.toml file to the runner.

1. Navigate to your project repository.

1. Select + > New file

1. In the filename, type config.toml.

1. Copy the config.toml from your job output into the toml file you created in your repository (make sure to replace the your-token value with your runner token instead).

Your config.toml will look something like this:

```
toml
concurrent = 1
checkinterval = 0
connectionmaxage = "15m0s"
shutdowntimeout = 0
[sessionserver]
  sessiontimeout = 1800
[[runners]]
  name = "docker-runner"
  url = "https://gitlab.com"
  id = 40174213
  token = "your-token"
  tokenobtainedat = 2025-05-24T12:10:22Z
  tokenexpiresat = 0001-01-01T00:00:00Z
  executor = "docker"
[runners.custombuildldir]
[runners.cache]
  MaxUploadedArchiveSize = 0
[runners.cache.s3]
[runners.cache.gcs]
[runners.cache.azure]
[runners.docker]
  tlsverify = false
  image = "alpine:latest"
  privileged = false
  disableentrypointoverwrite = false
  oomkilldisable = false
  disablecache = false
  volumes = ["/cache"]
```

```
shmsize = 0
networkmtu = 0
```

1. Before saving your config.toml file, update the following fields in your config.toml:

```
toml
privileged = true
volumes = ["/certs/client", "/cache"]
```

1. Commit your config.toml file.

1. Select your .gitlab-ci.yml file.

1. Select Edit > Edit in pipeline editor.

1. After the gitlab-runner register command, add the following into the script:

```
yml
- scp config.toml root@$ip:/etc/gitlab-runner/config.toml
- ssh root@$ip 'gitlab-runner restart'
```

1. After doing this, you will have the following .gitlab-ci.yml file:

```
yml
stages:
  - deploy
```

deploy config:

stage: deploy

image: ubuntu:latest

beforescript:

```
- 'which ssh-agent || ( apt-get update -y && apt-get install openssh-client git -y )'
- eval $(ssh-agent -s)
- chmod 400 "$SSHPRIVATEKEY"
- ssh-add "$SSHPRIVATEKEY"
- mkdir -p ~/.ssh
- chmod 700 ~/.ssh
- ssh-keyscan -t rsa,ed25519 $ip >> ~/.ssh/knownhosts
```

script:

```
- ssh root@$ip 'gitlab-runner unregister --all-runners'
- ssh root@$ip 'gitlab-runner register --non-interactive --url
```

```
https://lt.gitlabtraining.cloud --executor "docker" --docker-image alpine:latest --token
your-runner-token'
```

```
- scp config.toml root@$ip:/etc/gitlab-runner/config.toml
- ssh root@$ip 'gitlab-runner restart'
- ssh root@$ip 'cat /etc/gitlab-runner/config.toml'
```

1. Select Commit changes.

This script copies your configuration to the runner machine. When the runner is registered, the new configuration is applied!

Task F. Testing the Runner

To test the runner, let's create a basic Docker in Docker configuration to use for a project.

1. Navigate to your CI/CD Runner project.

1. Start by disabling instance level runners to ensure the jobs run on your project runner. To do this, navigate to Settings > CI/CD.

1. Click the arrow next to Runners to expand the Runners section.

1. Toggle Enable instance runners for this project off.

1. Navigate back to your code repository.

1. Select + > New file.

1. For the filename, type Dockerfile. Add the following content:

```
Docker
FROM node:latest

WORKDIR /app
CMD ["npm", "start"]
```

1. Select Commit changes.

1. Select your .gitlab-ci.yml file.

1. Select Edit > Edit in pipeline editor.

1. Delete all of your existing jobs and stages, leaving you with an empty file.

1. Add a build stage:

```
yml
stages:
  - build
```

1. Add the following build job:

```
yml
```

```

build image:
  stage: build
  image: docker:27
  services:
    - docker:27-dind
  variables:
    IMAGE: $CI_REGISTRYIMAGE/$CICOMMITREFSLUG:$CICOMMITSHA
  script:
    - docker login -u $CI_REGISTRYUSER -p $CI_REGISTRYPASSWORD $CI_REGISTRY
    - docker build -t $IMAGE .
    - docker push $IMAGE

```

1. Select Commit changes.

When your pipeline runs, you should see your job succeed using your new Docker runner!

Task G. Disabling the runner

For future labs, it is best to use the instance runners to ensure consistency in your results. To allow for this, disable your custom runner using the following steps:

1. Navigate to your project.
1. Select Settings > CI/CD.
1. Click the arrow next to Runners to expand the Runners section.
1. Toggle Enable instance runners for this project to on.
1. Pause your assigned project runner by selecting the pause button.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab Advanced CI/CD, please submit your changes via Merge Request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/advgitlabci cdhandsonlab2.md

The purpose of this lab is to demonstrate the benefits of caching in a GitLab CI/CD Pipeline. A cache is one or more files a job downloads and saves. Subsequent jobs that use the same cache don't have to download the files again, so they execute more quickly. Caches, unlike artifacts, are not stored in GitLab.

> Estimated time to complete: 15 minutes

Objectives

The objectives for this lab are:

- Displaying the benefits of caching

Task A. Building a basic pipeline

To start, let's create a basic Node.js application to use for demonstration of pipeline builds.

1. Navigate to your ILT group, named "My Test Group - <your-username>".

1. Select New project.

1. Select Create blank project.

1. Set the project name to Node, leave all other options as default, then select Create project.

1. In this project, select + > New file.

1. Set the filename to index.js and add the following code:

```
js
module.exports.binarySearch = function binarySearch(arr, val) {
  let start = 0;
  let end = arr.length - 1;
  while (start <= end) {
    let mid = Math.floor((start + end) / 2);
    if (arr[mid] === val) {
      return mid;
    }
    if (val < arr[mid]) {
      end = mid - 1;
    } else {
      start = mid + 1;
    }
  }
  return -1;
}
```

```
module.exports.linearSearch = function linearSearch(arr, val){
  let index = 0;
  let found = false;
  while (!found && index < arr.length){
    if (arr[index] == val){
      found = true;
    }else{
```

```

        index += 1;
    }
}

if (!found){
    index = -1;
}

return index;
}

```

> This code is a simple binary search, which will search a sorted array for a value, returning either the index of the value if it exists in the array, or -1 if the value is not found.

1. Select Commit changes.

To create a Node project, we need to also create a package.json file.

1. Navigate to your project repository.

1. Select + > New file.

1. Set the Filename to package.json and add the following text:

```

json
{
  "name": "ci-cd-demos",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC"
}

```

1. Select Commit changes.

Task B. Creating tests

To demonstrate the concepts of caching, let's introduce some testing into our application. First, let's create some tests for the binary search.

1. Navigate to your code repository.

1. Select + > New file.

1. Set the filename to binarysearch.test.js and add the following code:

```
js
const {binarySearch} = require("./index.js");

describe("Binary search tests", () => {
  test("Search should succeed on first element", () => {
    expect(binarySearch([1,2,3,4],1)).toBe(0);
  });

  test("Search should succeed on last element", () => {
    expect(binarySearch([1,2,3,4],4)).toBe(3);
  });

  test("Search should succeed on any element", () => {
    expect(binarySearch([1,2,3,4],2)).toBe(1);
  });

  test("Search should return -1 on not found", () => {
    expect(binarySearch([1,2,3,4],10)).toBe(-1);
  });
});
```

1. Select Commit changes.

1. We will create a similar set of tests for the linear search. Create a new file named linearsearch.test.js and add the following code:

```
js
const {linearSearch} = require("./index.js");

describe("Linear search tests", () => {
  test("Search should succeed on first element", () => {
    expect(linearSearch([1,2,3,4],1)).toBe(0);
  });

  test("Search should succeed on last element", () => {
    expect(linearSearch([1,2,3,4],4)).toBe(3);
  });

  test("Search should succeed on any element", () => {
    expect(linearSearch([1,2,3,4],2)).toBe(1);
  });

  test("Search should return -1 on not found", () => {
    expect(linearSearch([1,2,3,4],10)).toBe(-1);
  });
});
```

```
});
```

1. Commit your linearsearch.test.js file.

Next, we will create a .gitlab-ci.yml file to define our tests.

1. Navigate to the project repository.
1. Select + > New file.
1. For the filename, enter .gitlab-ci.yml.
1. Add the following job definitions to the file:

```
yaml
stages:
  - test

default:
  image: node:latest

test binarysearch:
  stage: test
  script:
    - npm install jest
    - nodemodules/.bin/jest binarysearch.test.js

test linearsearch:
  stage: test
  script:
    - npm install jest
    - nodemodules/.bin/jest linearsearch.test.js
```

1. Select Commit changes.

Task C. Using caching to optimize jobs

Let's take a look at this set of job definitions to see if they can be made more efficient. One common optimization we can look for is repetition in scripts run during jobs. In this example, both jobs need to install the jest-junit package with npm. Rather than installing jest-junit twice, we can cache the package between jobs.

1. In your .gitlab-ci.yml file, add the following job:

```
yaml
install deps:
  stage: deps
  script:
```



```
- npm install jest
cache:
  key: $CICOMMITREFSLUG
  paths:
    - nodemodules
```

1. Next, add the deps stage before the test stage to ensure the dependencies are cached before the testing scripts run:

```
yml
stages:
  - deps
  - test
```

The .gitlab-ci.yml file should now look like this:

```
yml
stages:
  - deps
  - test

default:
  image: node:latest

install deps:
  stage: deps
  script:
    - npm install jest
  cache:
    key: $CICOMMITREFSLUG
    paths:
      - nodemodules

test binarysearch:
  stage: test
  script:
    - npm install jest
    - nodemodules/.bin/jest binarysearch.test.js

test linearsearch:
  stage: test
  script:
    - npm install jest
    - nodemodules/.bin/jest linearsearch.test.js
```

> With this definition, we create a cache with a key that matches the CICOMMITREFSLUG. This ensures that each job will receive a unique cache. The data being cached is the nodemodules

folder. To set up the cache for use, we use the script to run the install command for the jest package, which we will use to run the tests.

Now that we have a cache defined, we can remove the jest package install from each job.

1. Remove the npm install jest commands from your jobs and replace it with a cache reference. Below is an example of the completed .gitlab-ci.yml file:

```
yml
stages:
  - deps
  - test

default:
  image: node:latest

install deps:
  stage: deps
  script:
    - npm install jest
  cache:
    key: $CI_COMMITREFSLUG
    paths:
      - nodemodules

test binarysearch:
  stage: test
  script:
    - nodemodules/.bin/jest binarysearch.test.js
  cache:
    key: $CI_COMMITREFSLUG
    paths:
      - nodemodules

test linearsearch:
  stage: test
  script:
    - nodemodules/.bin/jest linearsearch.test.js
  cache:
    key: $CI_COMMITREFSLUG
    paths:
      - nodemodules
```

1. Commit your changes to main.

1. Navigate to your pipeline and verify that your testing jobs now run successfully.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab Advanced CI/CD, please submit your changes via Merge Request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/advgitlabci cdhandsonlab3.md

The goal of this lab is to explore the different ways that we can configure testing in an application.

> Estimated time to complete: 15 minutes

Objectives

- Stop a pipeline after failure
- Unit test reporting
- Parallelization

In this lab, we will explore the different ways that we can configure testing in an application. Currently, we have the following testing setup for our project:

```
yaml
stages:
  - deps
  - test

default:
  image: node:latest

install deps:
  stage: deps
  script:
    - npm install jest
  cache:
    key: $CICOMMITREFSLUG
    paths:
      - nodemodules

test binarysearch:
  stage: test
  script:
    - nodemodules/.bin/jest binarysearch.test.js
  cache:
    key: $CICOMMITREFSLUG
    paths:
```

- nodemodules

test linearsearch:

stage: test

script:

- nodemodules/.bin/jest linearsearch.test.js

cache:

key: \$CICOMMITREFSLUG

paths:

- nodemodules

This lab will explore how we can ensure a test pipeline when a single job fails. We will also see how we can add test reporting to our test jobs.

Task A. Stopping a Pipeline after Failure

In this example, let's look at how we can cancel the pipeline in the case where one of our tests fails.

1. Navigate to your Node project repository.

1. Select .gitlab-ci.yml.

1. Select Edit > Edit in pipeline editor.

1. Just below the stages section of your .gitlab-ci.yml file, add a workflow to auto cancel the jobs.

```
yml
workflow:
  autocancel:
    onjobfailure: all
```

1. With this configuration, if any jobs fail, the whole pipeline will fail. To test this, you can create a test job that will purposely fail.

```
yml
test fail:
  stage: test
  script:
    - jet test.js
```

1. Select Commit changes.

Let's see how the pipeline handles the failed job.

1. In the left sidebar, select Build > Pipelines.

1. Select your most recent pipeline and observe the jobs. Note that when the test fail job fails, other jobs cancel, showing a grey slash icon.

Now that we have verified the auto cancel works, let's remove the failing job.

1. Navigate to your repository.

1. Select .gitlab-ci.yml.

1. Select Edit > Edit in pipeline editor.

1. Remove the test fail job. Your .gitlab-ci.yml file will look like this:

```
yml
stages:
  - deps
  - test

workflow:
  autocancel:
    onjobfailure: all

default:
  image: node:latest

install deps:
  stage: deps
  script:
    - npm install jest
  cache:
    key: $CI_COMMITREFSLUG
    paths:
      - nodemodules

test binarysearch:
  stage: test
  script:
    - nodemodules/.bin/jest binarysearch.test.js
  cache:
    key: $CI_COMMITREFSLUG
    paths:
      - nodemodules

test linearsearch:
  stage: test
  script:
    - nodemodules/.bin/jest linearsearch.test.js
  cache:
    key: $CI_COMMITREFSLUG
    paths:
```

- nodemodules

1. Select Commit changes.

Task B. Adding Test Reports

In this task, we will add a test report to our test jobs.

1. Ensure you are still in the Pipeline Editor (if not, navigate to Build > Pipeline Editor).

1. We are going to adjust our jest commands for the test binarysearch and test linearsearch jobs to add a testResultsProcessor to the command. We can do this by adding the --ci --testResultsProcessor=jest-junit flags to the command. The --ci flag will make Jest assume it is running in a CI environment. For this to work we also have to install jest-junit by adding it to our install deps. Below is an example of the jobs after the changes have been made:

```
yaml
install deps:
  stage: deps
  script:
    - npm install jest jest-junit
  cache:
    key: $CICOMMITREFSLUG
    paths:
      - nodemodules
```

```
test binarysearch:
  stage: test
  script:
    - nodemodules/.bin/jest --ci --testResultsProcessor=jest-junit binarysearch.test.js
  cache:
    key: $CICOMMITREFSLUG
    paths:
      - nodemodules
```

```
test linearsearch:
  stage: test
  script:
    - nodemodules/.bin/jest --ci --testResultsProcessor=jest-junit linearsearch.test.js
  cache:
    key: $CICOMMITREFSLUG
    paths:
      - nodemodules
```

1. The test results need to be stored in a JUnit file in order to be accessed by the pipeline. To do so, we need to add the following code snippet to both of our tests after the script keyword:

```
yaml
artifacts:
  when: always
  reports:
    junit: junit.xml
```

The tests will now look like this:

```
yaml
test binarysearch:
  stage: test
  script:
    - nodemodules/.bin/jest --ci --testResultsProcessor=jest-junit binarysearch.test.js
  artifacts:
    when: always
    reports:
      junit: junit.xml
  cache:
    key: $CICOMMITREFSLUG
    paths:
      - nodemodules
```

```
test linearsearch:
  stage: test
  script:
    - nodemodules/.bin/jest --ci --testResultsProcessor=jest-junit linearsearch.test.js
  artifacts:
    when: always
    reports:
      junit: junit.xml
  cache:
    key: $CICOMMITREFSLUG
    paths:
      - nodemodules
```

1. After making these changes, select Commit changes.
1. In the left sidebar, select Build > Pipelines.
1. Select your most recent pipeline.
1. Wait for the tests to complete. Refresh after the test jobs are complete and select the tab Tests.
1. You will see a report of your test results in the tab.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab Advanced CI/CD, please submit your changes via Merge Request!

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/advgitlabci cdhandsonlab4.md

The goal of this lab is to use things like hidden jobs and map merges to help make your code more concise, and avoid repetition.

> Estimated time to complete: 15 minutes

Objectives

- Reducing repetition in your pipeline
- Hidden jobs and map merges

In our testing pipeline, there are a few instances where we are repeating code and definitions. We already saw that it was possible to use caches to reduce repetition in the npm install of packages. In this lab, you will learn how to further reduce the repetition in your code.

Here is the current .gitlab-ci.yml file definition:

yml:

stages:

- deps
- test

workflow:

autocancel:

onjobfailure: all

default:

image: node:latest

install deps:

stage: deps

script:

- npm install jest jest-junit

cache:

key: \$CICOMMITREFSLUG

paths:

- nodemodules

test binarysearch:


```

stage: test
script:
  - nodemodules/.bin/jest --ci --testResultsProcessor=jest-junit binarysearch.test.js
artifacts:
  when: always
  reports:
    junit: junit.xml
cache:
  key: $CICOMMITREFSLUG
  paths:
    - nodemodules

```

```

test linearsearch:
  stage: test
  script:
    - nodemodules/.bin/jest --ci --testResultsProcessor=jest-junit linearsearch.test.js
  artifacts:
    when: always
    reports:
      junit: junit.xml
  cache:
    key: $CICOMMITREFSLUG
    paths:
      - nodemodules

```

Task A. Simplifying your jobs

1. Navigate to your project repository.

1. Select the .gitlab-ci.yml file.

1. Select Edit > Edit in pipeline editor.

1. In our current .gitlab-ci.yml file, all of the tests contain the same set of artifacts. Instead of defining the artifacts at each job, you can instead first create a hidden job with the definition:

```

.yml
.artifactdef: &artifactdef
  artifacts:
    when: always
    reports:
      junit: junit.xml

```

1. You can then use a map merge to add the artifact definition to all of your testing jobs:

```

.yml
test binarysearch:

```

```

stage: test
script:
  - nodemodules/.bin/jest --ci --testResultsProcessor=jest-junit binarysearch.test.js
<<: artifactdef
cache:
  key: $CICOMMITREFSLUG
  paths:
    - nodemodules

```

```

test linearsearch:
  stage: test
  script:
    - nodemodules/.bin/jest --ci --testResultsProcessor=jest-junit linearsearch.test.js
  <<: artifactdef
  cache:
    key: $CICOMMITREFSLUG
    paths:
      - nodemodules

```

1. Additionally, we can move the cache into a hidden job to avoid repetition. In this example, we moved these definitions into the cachedef hidden job:

```

yaml
stages:
  - deps
  - test

workflow:
  autocancel:
    onjobfailure: all

default:
  image: node:latest

.artifactdef: &artifactdef
  artifacts:
    when: always
    reports:
      junit: junit.xml

.cachedef: &cachedef
  cache:
    key: $CICOMMITREFSLUG
    paths:
      - nodemodules

install deps:
  stage: deps
  script:

```

```
- npm install jest jest-junit
<<: cachedef
```

test binarysearch:

```
stage: test
```

```
script:
```

```
- nodemodules/.bin/jest --ci --testResultsProcessor=jest-junit binarysearch.test.js
```

```
<<: [artifactdef, cachedef]
```

test linearesearch:

```
stage: test
```

```
script:
```

```
- nodemodules/.bin/jest --ci --testResultsProcessor=jest-junit linearesearch.test.js
```

```
<<: [artifactdef, cachedef]
```

> This change not only reduces the total number of lines of code, but also makes it so if the artifact changes, you only need to change it in one place, rather than multiple locations.

1. After making these changes, select Commit changes.

1. Monitor your pipeline by selecting Build > Pipelines in the left sidebar.

1. After verifying that your pipeline ran successfully, return to your .gitlab-ci.yml file. Can you find any other optimizations for this file?

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab Advanced CI/CD, please submit your changes via Merge Request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/advgitlabci cdhandsonlab5.md

In this lab we will analyze more complex merge processes, looking specifically at merge trains and merge conflicts. First, we will start with merge trains.

> Estimated time to complete: 15 minutes

Objectives

- Understand the concept of merge trains and their benefits
- Enable merge trains in a GitLab project
- Create and run a merge train

- Identify and resolve merge conflicts in GitLab
- Practice handling conflicting merge requests

Task A. Enabling Merge Trains

1. To enable a merge train in your project, in the left sidebar, select Settings > Merge requests.
1. Under Merge options, click the options Enable merged results pipeline and Enable merge trains.
1. Scrolling down the page slightly to the Merge Checks section, click the option Pipelines must succeed
1. At the bottom of the section, select Save changes.

Task B. Running a merge train

To demonstrate a merge train, let's create a purposefully long CI/CD job.

1. Navigate to your project repository.
1. Select the .gitlab-ci.yml file.
1. In your existing CI/CD file, add the following job to your pipeline:

```
yml
pause:
  stage: test
  script:
    - sleep 4m
```

1. Add in the following rules to ensure jobs run on merge request pipelines:

```
yml
workflow:
  autocancel:
    onjobfailure: all
  rules:
    - if: $CIPIPELINESOURCE == 'mergerequestevent'
    - if: $CICOMMITBRANCH == $CIDEFAULTBRANCH
```

1. Commit these changes.

The current pipeline should look like this:

```
yml
stages:
```

- deps
- test

workflow:

autocancel:

onjobfailure: all

rules:

- if: \$CIPIPELINESOURCE == 'mergerequestevent'
- if: \$CICOMMITBRANCH == \$CIDEFAULTBRANCH

default:

image: node:latest

.artifactdef: &artifactdef

artifacts:

when: always

reports:

junit: junit.xml

.cachedef: &cachedef

cache:

key: \$CICOMMITREFSLUG

paths:

- nodemodules

install deps:

stage: deps

script:

- npm install jest jest-junit

<<: cachedef

test binarysearch:

stage: test

script:

- nodemodules/.bin/jest --ci --testResultsProcessor=jest-junit binarysearch.test.js

<<: [artifactdef, cachedef]

test linearsearch:

stage: test

script:

- nodemodules/.bin/jest --ci --testResultsProcessor=jest-junit linearsearch.test.js

<<: [artifactdef, cachedef]

pause:

stage: test

script:

- sleep 4m

1. Select Commit changes to update your .gitlab-ci.yml file.

Adding this job will ensure that you have enough time to create two merge requests.

To start, create your two merge requests. For the first merge request:

1. Select Code > Branches.
1. Select New branch.
1. Add the branch name train.
1. Leave all other options as default and select Create branch.
1. Select the README.md file and add some changes to it.
1. Select Create merge request.
1. Leave all options as default and select Create merge request.

For the second merge request:

1. Select Code > Branches.
1. Select New branch.
1. Add the branch name train-2.
1. Leave all other options as default and select Create branch.
1. Select the README.md file and add some changes to it. Try making different changes to the file than the ones you did in the previous branch.
1. Select Create merge request.
1. Leave all options as default and select Create merge request.

Now that both merge requests have been created:

1. Set them both to auto-merge. You will see a message stating Set by your user to start a merge train when all merge checks pass.
1. Await the completion of your merge requests and verify that they merge successfully.

Task C. Merge Conflicts

When multiple users work on a project at the same time, merge conflicts are often inevitable. In this lab, you will learn how to handle merge requests in your project.

1. Remove the pause job from your CI/CD project to avoid slowdowns. Right now, your file will look like this:

```

yml
stages:
  - deps
  - test

workflow:
  autocancel:
    onjobfailure: all

default:
  image: node:latest

.artifactdef: &artifactdef
artifacts:
  when: always
  reports:
    junit: junit.xml

install deps: &cachedef
stage: deps
script:
  - npm install jest jest-junit
<<: cachedef

test binarysearch:
stage: test
script:
  - nodemodules/.bin/jest --ci --testResultsProcessor=jest-junit binarysearch.test.js
<<: [artifactdef, cachedef]

test linearsearch:
stage: test
script:
  - nodemodules/.bin/jest --ci --testResultsProcessor=jest-junit linearsearch.test.js
<<: [artifactdef, cachedef]

```

Now, let's create two merge requests that conflict:

1. Select Code > Branches.
1. Select New branch.
1. Add the branch name conflict.
1. Leave all other options as default and select Create branch.
1. Select the index.js file. At the top of the file, add a comment to describe the function. An example comment is below.

```

js
// This method will create a binary search finding the value in list in log(n) time
module.exports.binarySearch = function binarySearch(arr, val) {
  let start = 0;
  let end = arr.length - 1;
  while (start <= end) {
    let mid = Math.floor((start + end) / 2);
    if (arr[mid] === val) {
      return mid;
    }
    if (val < arr[mid]) {
      end = mid - 1;
    } else {
      start = mid + 1;
    }
  }
  return -1;
}

```

1. Commit this code to the branch and create a new merge request from it. After you do this, create a new branch:

1. Select Code > Branches.

1. Select New branch.

1. Add the branch name conflict-2.

1. Leave all other options as default and select Create branch.

1. Select the index.js file. At the top of the file, add a different comment to describe the function. An example comment is below:

```

js
//A binary search will search a list in log(n) time
module.exports.binarySearch = function binarySearch(arr, val) {
  let start = 0;
  let end = arr.length - 1;
  while (start <= end) {
    let mid = Math.floor((start + end) / 2);
    if (arr[mid] === val) {
      return mid;
    }
    if (val < arr[mid]) {
      end = mid - 1;
    } else {
      start = mid + 1;
    }
  }
}

```



```
    return -1;
}
```

1. Commit this code to the branch and create a new merge request from it.

1. Return to your conflict merge request. Select the arrow next to the merge button then select merge immediately and merge it into the repository.

1. After it merges, navigate to your conflict-2 merge request. You will now see that the merge is blocked, stating Merge conflicts must be resolved.

1. Select the option Resolve conflicts. You will have the option to select either using the code in the current merge request, or using the code in main.

1. Select your preferred option, then select Commit to source branch.

After doing this, you will now be able to merge your merge request.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab Advanced CI/CD, please submit your changes via Merge Request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/advgitlabci cdhandsonlab6.md

The purpose of this lab is to create a review app from a Node